

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Technical Presentation Questions

Course Code:	CSA0315	Course Title:	Data Structures
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Technical Presentations Questions
Weightage:	10% of CIA	Total Marks:	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays&Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	Nandha Krishna V	Reg. No.:	192524012
Section / Batch:		Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 3 Concept mapping questions. Total: 100 Marks.
- When a student answer using a concept map, in evaluation the answer should contain following four requirements: Understanding of concepts, Connections between ideas, Logical structure, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- Create a PPT with 15 slides – Laptop permitted for presentation in the class. Take Print out and submit in the class.

Section / Topic	Focus	Q Nos	Marks
Realization of ADT in 'C' – Array DS- Performance analysis and Measurement: Time and space analysis of algorithms.	Linked List data Structure	1	30
Primitive and non-primitive data structure, Linear and non-linear data structure Arrays	Tries data Structure	2	40
Dynamic linear Data Structures	Queue Data structures	3	40
GRAND TOTAL:			100 Marks

Annexure A – Technical Presentation

- 1) A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation
- 2) A dictionary application needs to quickly search whether a word exists in its database. Suggest an appropriate data structure and justify based on search performance. Prepare a technical Presentation.
- 3) A task scheduling system assigns jobs based on execution priority levels. Suggest a suitable data structure and justify performance efficiency.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

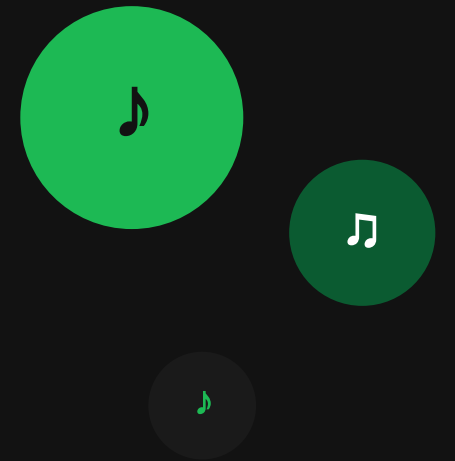
DATA STRUCTURES & ALGORITHMS

Dynamic Playlist Management

Why the Linked List is the Optimal Choice for a Music Streaming App's Playlist

Technical Presentation | Add • Remove • Reorder • Play

Presented by
NANDHA KRISHNA V
192524012



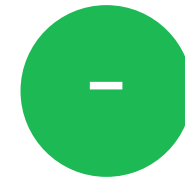
The Challenge: Dynamic Playlist Operations

A playlist is never static — a good data structure must handle constant change gracefully



Add Songs

Insert a new track at the end, beginning, or right after the currently playing song.



Remove Songs

Delete any track instantly — a skipped song or one the user no longer wants.



Reorder / Shuffle

Move a song to a new position, or shuffle the entire queue on demand.



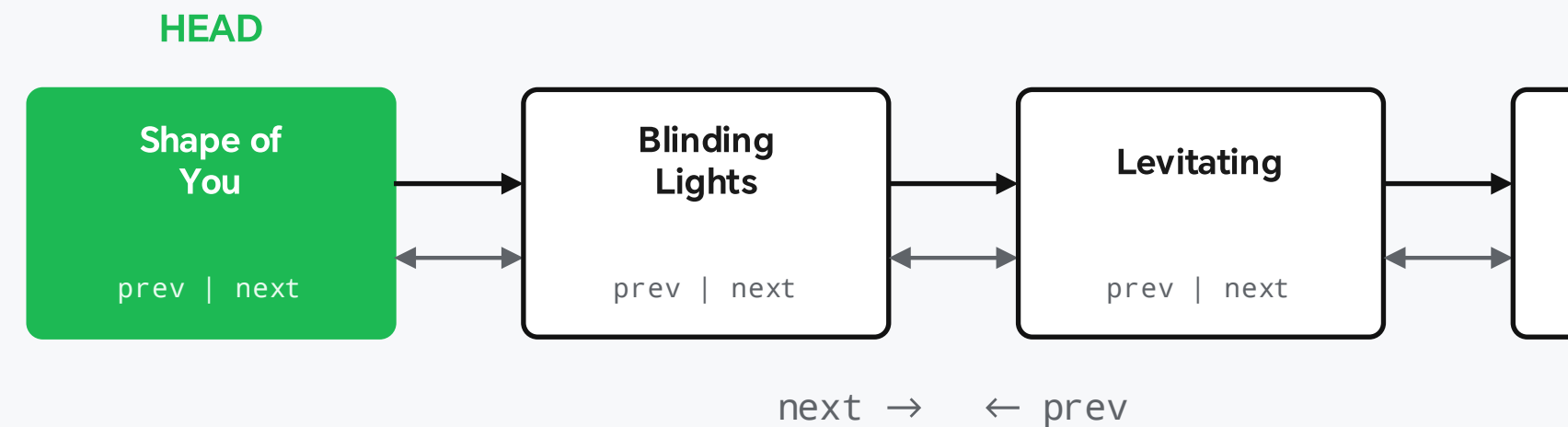
Sequential Playback

Move to the next or previous track continuously as the user listens.

The Solution: Doubly Linked List

Each song is a node — no shifting, no fixed size, no wasted memory

- **Node** holds Song Data (title, artist, duration) + a pointer to the Next node + a pointer to the Previous node.
- **Head & Tail** pointers give instant access to the first and last song without traversal.
- **Bidirectional links** let playback move to next/previous track in $O(1)$ time.
- **Optional circular link** connects tail back to head, enabling seamless "Repeat / Loop" playback.



Dashed idea: tail.next → head keeps the queue looping for Repeat Mode

Efficiency Analysis: Array vs Linked List

Time complexity of core playlist operations

Operation	Array / ArrayList	Linked List (DLL)
Insert at beginning	$O(n)$ — shifts all elements	$O(1)$
Insert / delete in middle	$O(n)$ — shifts elements	$O(1)$ — with node reference
Delete a song	$O(n)$	$O(1)$
Reorder / move a song	$O(n)$ — re-index needed	$O(1)$ — relink pointers
Access by index	$O(1)$	$O(n)$ — sequential access
Memory allocation	Fixed / contiguous	Dynamic, per-node

Key takeaway: Playlists are modified far more often than they are randomly indexed — playback is mostly sequential. The Linked List trades rarely-needed $O(1)$ random access for $O(1)$ insert/delete, exactly matching real playlist usage.

Implementation & Conclusion

A doubly linked list, mapped to real playlist operations

```
class Node:
    song, artist, duration
    prev -> Node
    next -> Node

class Playlist:
    head, tail

addSong(song, atEnd=True)      # O(1)
removeSong(node)               # O(1)
moveSong(node, newPos)        # O(1)
playNext() -> node.next        # O(1)
playPrevious() -> node.prev    # O(1)
```

- O(1) insertion and deletion at any position — no shifting of songs
- Dynamic size — grows and shrinks with the playlist, no wasted memory
- Bidirectional traversal — instant next/previous track navigation
- Circular variant enables seamless Repeat / Loop playback

Best Fit: the Doubly Linked List matches exactly how users actually build, edit, and play a playlist.

DATA STRUCTURES & ALGORITHMS

Fast Word Lookup for a Dictionary Application

Choosing the Right Data Structure, Justified by Search Performance

Technical Presentation | Search • Insert • Prefix Match



The Challenge: Searching a Huge Vocabulary

A dictionary app must answer “does this word exist?” instantly, at scale



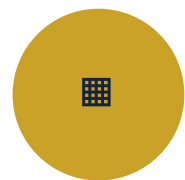
Instant Lookup

Check whether a typed word exists in the database with minimal delay, on every keystroke.



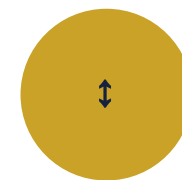
Prefix Search

Support autocomplete and “starts with...” queries as the user types, not just exact matches.



Large Scale

Store and search 100,000+ words efficiently without excessive memory overhead.



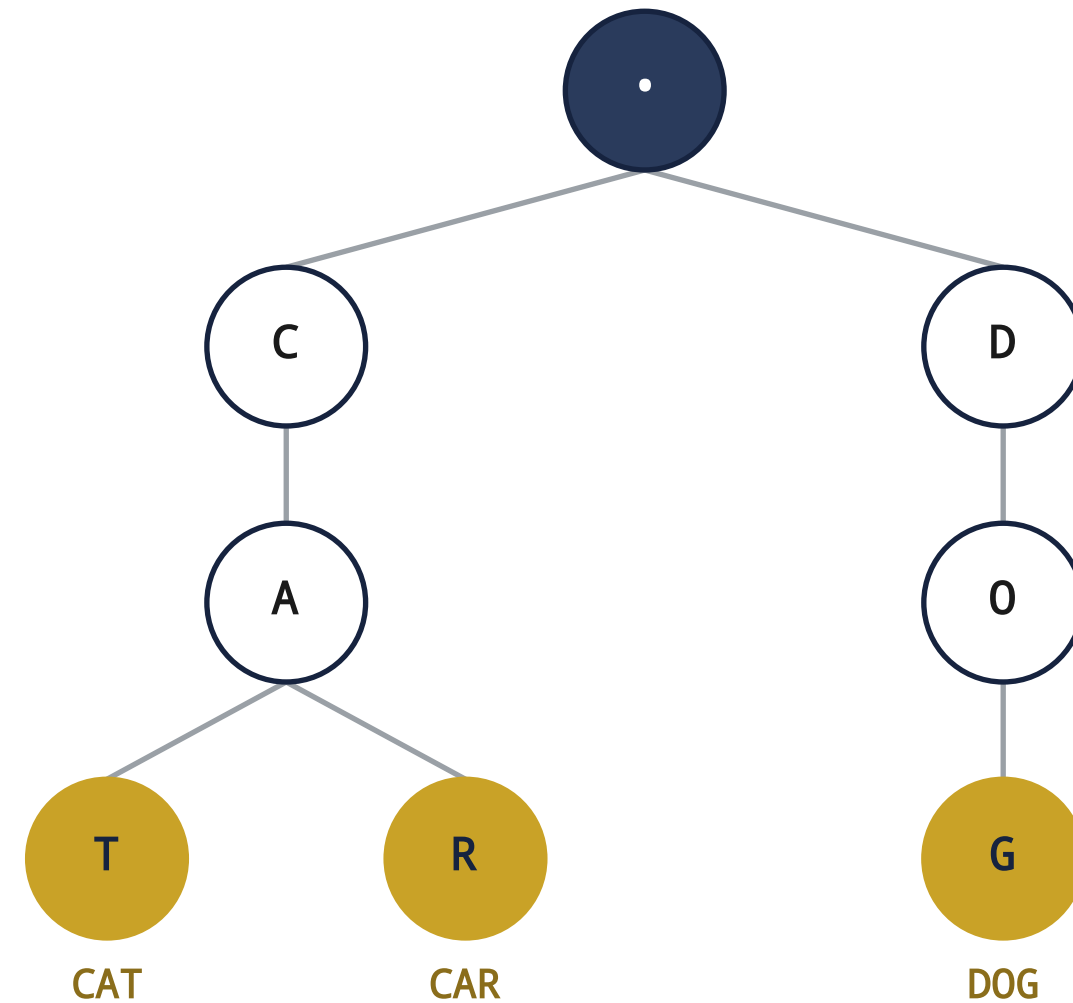
Ordered Access

Retrieve words in alphabetical order for spell-check and “did you mean” suggestions.

The Solution: Trie (Prefix Tree)

Words that share letters share the same path through the tree

- **Node** holds one character, a set of child pointers (one per possible next letter), and an `isEndOfWord` flag.
- **Shared prefixes** words like CAT and CAR reuse the same C → A path, saving memory across the vocabulary.
- **Search a word** walk one node per character — $O(L)$, where L is the word's length, not the dictionary size.
- **Prefix search** the same walk, stopped early, instantly powers autocomplete suggestions.



Gold nodes mark `isEndOfWord = True` — a complete word ends there

Efficiency Analysis: Search Performance

n = number of words in the dictionary, *L* = length of the word being searched

Structure	Search (word exists)	Prefix / Autocomplete	Memory Behaviour
Unsorted Array / List	$O(n)$ — scan every word	$O(n \times L)$ — scan + compare	Low overhead, no sharing
Balanced BST (sorted)	$O(\log n \times L)$	$O(\log n + k)$ — extra logic	Moderate, pointer-heavy
Hash Table	$O(L)$ average	Not supported natively	Extra space for hash buckets
Trie (Prefix Tree)	$O(L)$ — independent of <i>n</i>	$O(L + k)$ — native support	Shared prefixes save space

Key takeaway: Trie search time depends only on word length (*L*), not on how many words are stored — lookup stays fast even at a million entries. It is also the only structure here with native prefix support, which a hash table cannot offer and a BST only approximates.

Implementation & Conclusion

A Trie, mapped to the dictionary's core operations

```
class TrieNode:
    children[26]          # or map<char, Node>
    isEndOfWord = False

class Trie:
    root = TrieNode()

    insert(word):          # O(L)
        node = root
        for ch in word:
            node = node.children[ch]
                    or new TrieNode()
        node.isEndOfWord = True

    search(word) -> bool:   # O(L)
        node = walk(word)
        return node and node.isEndOfWord

    startsWith(prefix) -> bool: # O(L)
        return walk(prefix) != null
```

- O(L) search — independent of dictionary size, even with millions of words
- Built-in prefix matching — powers autocomplete and “did you mean” suggestions
- Memory-efficient — shared prefixes reduce redundant storage
- Naturally sorted — a depth-first walk returns words in alphabetical order

Best Fit: the Trie delivers instant lookup and native prefix search that arrays and hash tables can't match.

DATA STRUCTURES & ALGORITHMS

Efficient Task Scheduling by Priority

Choosing the Right Data Structure, Justified by Performance Efficiency

Technical Presentation | Insert • Extract-Max • Reprioritize



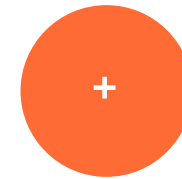
The Challenge: Scheduling Jobs by Priority

The scheduler must always know — instantly — which job to run next



Fast Top-Priority Access

Identify and dispatch the highest-priority job in the queue without scanning everything.



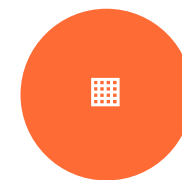
Frequent Insertions

New jobs arrive continuously at arbitrary priority levels and must be added efficiently.



Dynamic Reprioritization

A job's priority can change mid-queue (e.g. aging, priority boost) and must re-settle quickly.



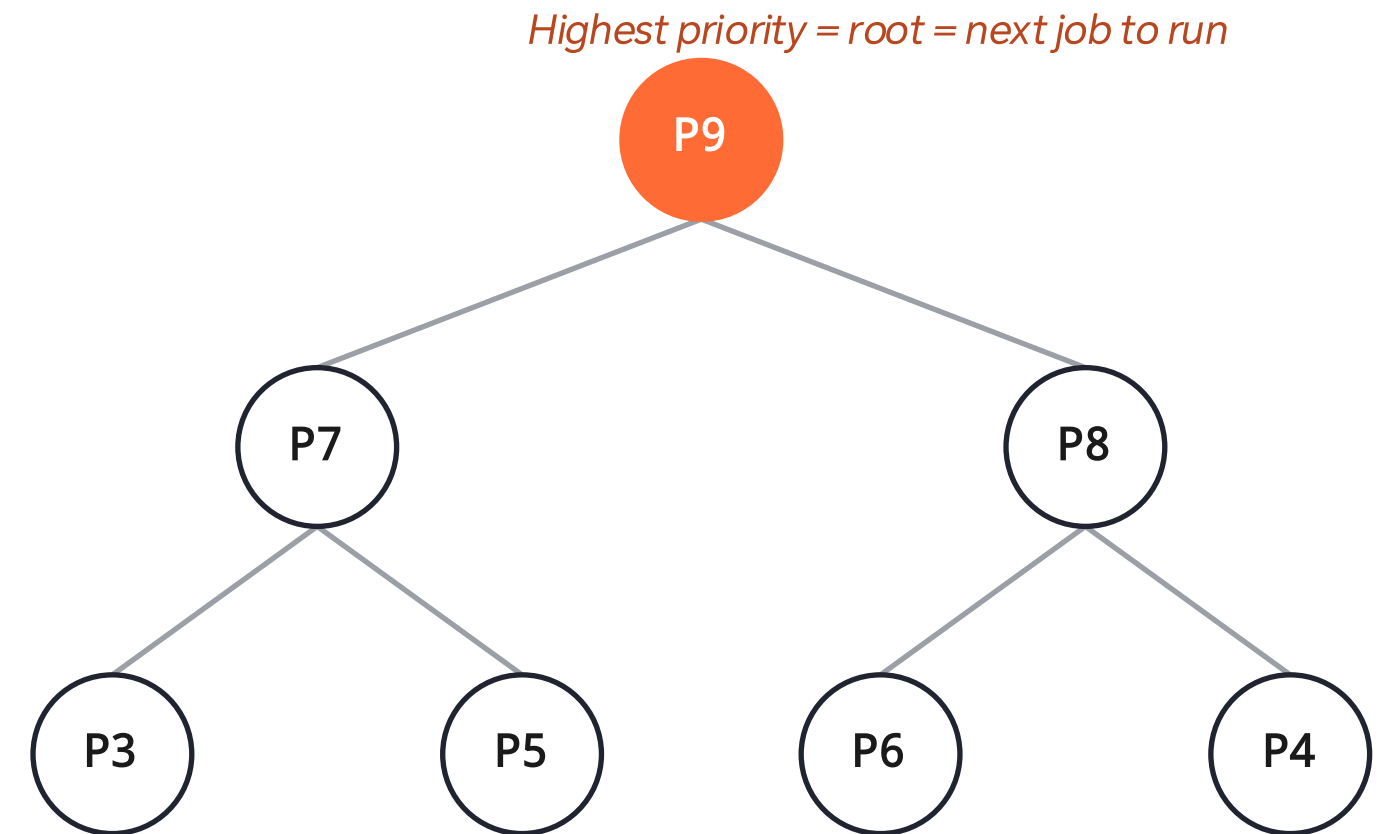
High Throughput

Handle thousands of concurrent jobs with minimal overhead per scheduling decision.

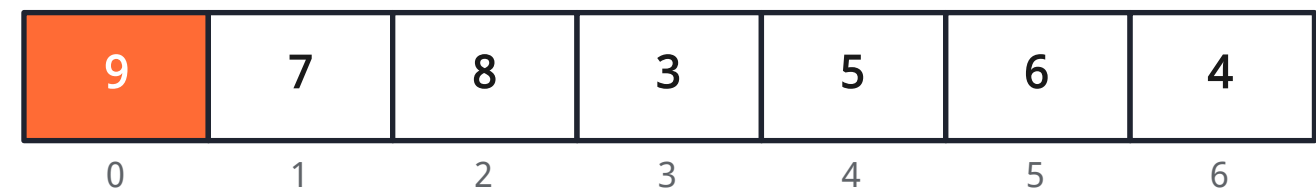
The Solution: Binary Heap (Priority Queue)

A complete binary tree stored in an array — the highest priority always sits at the root

- **Heap property** every parent's priority is $\geq$ its children's — so the most urgent job is always at the root.
- **Array-backed** no pointers needed; a node at index i has children at $2i+1$ and $2i+2$ — compact and cache-friendly.
- **Insert a job** append at the end, then “bubble up” until the heap property holds — $O(\log n)$.
- **Run next job** swap root with the last node, remove it, then “bubble down” — $O(\log n)$.



Array form:



$$\text{child}(i) = 2i+1, 2i+2 \quad \cdot \quad \text{parent}(i) = \lfloor (i-1)/2 \rfloor$$

Efficiency Analysis: Scheduling Structures

n = number of jobs currently waiting in the scheduler

Structure	Insert Job	Extract Highest Priority	Peek Next Job
Unsorted Array	$O(1)$	$O(n)$ — scan for max	$O(n)$
Sorted Array	$O(n)$ — shift to insert	$O(1)$ — remove from end	$O(1)$
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary Heap	$O(\log n)$	$O(\log n)$	$O(1)$

Key takeaway: A scheduler constantly alternates between adding new jobs and dispatching the next one — exactly the two operations a binary heap optimizes together. It gives balanced $O(\log n)$ insert/extract with instant $O(1)$ peek, using a simple array with no pointer overhead.

Implementation & Conclusion

A binary max-heap, mapped to the scheduler's core operations

```
class PriorityQueue:
    heap = []                # array-backed

    insert(job, priority):   # O(log n)
        heap.append((priority, job))
        bubbleUp(heap.length - 1)

    extractMax() -> job:     # O(log n)
        top = heap[0]
        heap[0] = heap[last]
        heap.removeLast()
        bubbleDown(0)
        return top

    peek() -> job:          # O(1)
        return heap[0]

    increaseKey(i, newPriority): # O(log n)
        heap[i].priority = newPriority
        bubbleUp(i)
```

- $O(\log n)$ insert & extract — scales smoothly to thousands of pending jobs
- $O(1)$ peek — the scheduler always knows the next job instantly
- Array-based storage — compact, cache-friendly, no pointer overhead
- increaseKey/decreaseKey support — handles priority aging and boosts cleanly

Best Fit: the Binary Heap always dispatches the highest-priority job next — fast, scalable, simple.